

User Management CRM Finish Roadmap

28/04/2026

One-hour coding bites to finish the app and make the frontend and backend consistent.

Roadmap

1. Normalize API Auth

Goal: Make all protected frontend requests send the JWT consistently.

- Create a small `apiFetch()` helper in `front-end/src/api/api.ts`.
- Automatically attach `Authorization: Bearer <token>`.
- Handle `401` responses in one predictable place.
- Replace raw lead `fetch()` calls with the helper.

Outcome: Lead create, edit, and delete calls work against protected backend routes.

2. Fix Backend Route Shape

Goal: Make backend routing match the app's feature list.

- Mount `usersRouter` in `back-end/src/routes/index.ts`.
- Keep `/auth/login` public.
- Protect `/auth/me`, `/leads`, and `/users`.
- Read authenticated user data from the JWT payload instead of `x-user-email`.

Outcome: API access rules are predictable and consistent.

3. Finish Users API

Goal: Replace the placeholder users backend route.

- Add `GET /users` to list users without passwords.
- Add `POST /users` to create users with hashed passwords.
- Optionally add `PATCH /users/:id` for role and status changes.
- Never return the `password` field from API responses.

Outcome: User management has a real backend foundation.

4. Swap Users Mock For Real API

Goal: Connect the users page to the backend.

- Replace `getUsers()` mock data with a real `/users` request.
- Keep or simplify the current typed response shape.
- Invalidate the users query after creating a user.
- Remove the random mock network failure.

Outcome: The users page reflects database state.

5. Make Lead Data Fetching Consistent

Goal: Move leads from manual state and effects to React Query.

- Add `getLeads`, `createLead`, `updateLead`, and `deleteLead` API functions.
- Use `useQuery` for the leads list.
- Use `useMutation` for create, edit, and delete actions.
- Invalidate `['leads']` after mutations.

Outcome: Leads and users follow the same frontend data pattern.

6. Add Shared Domain Contracts

Goal: Tighten consistency around data shapes.

- Align frontend `Lead` with the Prisma model.
- Make `company` `string | null` if it remains optional in the database.
- Align role values: backend uses `admin`, `salesAgent`, and `viewer`.
- Add Zod schemas for lead create and edit forms.

Outcome: Fewer hidden frontend/backend type mismatches.

7. Improve Backend Validation

Goal: Validate request bodies before database writes.

- Validate login input.
- Validate lead create and update input.
- Validate user create and update input.
- Return `400` for invalid input instead of generic `500`.

Outcome: API errors become intentional and easier to debug.

8. Add Basic Error Handling Middleware

Goal: Centralize backend error responses.

- Add Express error middleware.
- Reduce repeated `try/catch` blocks where reasonable.
- Keep Prisma `P2025` handling for not-found cases.
- Remove debug logs from auth and middleware.

Outcome: Routes become cleaner and responses become more consistent.

9. Add Minimal Frontend Auth Guard

Goal: Prevent protected pages from loading without a token.

- Add a route-level or layout-level guard.
- Redirect unauthenticated users to `/login`.
- Add a logout button in the layout.
- Clear the token on logout.

Outcome: Frontend navigation matches backend protection.

10. Polish CRUD UX

Goal: Make the UI feel complete without changing architecture.

- Add loading and saving states.
- Disable submit buttons while requests are running.
- Show form and server errors inline instead of using `alert()`.
- Close modals only after successful mutations.
- Add delete confirmation for destructive actions.

Outcome: The app behaves more like a real CRM.

11. Update Docs And Environment Examples

Goal: Make setup reproducible.

- Add backend `.env.example` with `DATABASE_URL`, `JWT_SECRET`, and `PORT`.
- Add frontend `.env.example` with `VITE_API_URL`.
- Update backend README route documentation.
- Update frontend README to reflect real leads and users integration.

Outcome: Another developer can run the project without reverse-engineering it.

12. Add Smoke Tests Or API Checks

Goal: Add enough verification to catch basic regressions.

- Check `/health` .
- Check login.
- Check authenticated `/leads` .
- Check authenticated `/users` .
- At minimum, run the frontend and backend builds.

Outcome: Future changes are less likely to silently break core flows.

Suggested Session Order

Session 1: API auth helper and lead JWT calls.

Session 2: Mount and implement the users backend.

Session 3: Replace the users mock with the real API.

Session 4: Convert leads to React Query.

Session 5: Add auth guards and logout.

Session 6: Add validation, docs, and UX polish.